Zephyrus Retail OS

Designing an Autonomous Modular Smart-City Retail Infrastructure

PROJECT REPORT

Course: Object Oriented Programming (IT620)

Group Name: Abstract Minds

Group Members

Enrollment Id	Name
2025121059	Jay Shah
202512100	Priya J Shah
202512105	Priya D Shah
202512110	Jenil Shah

1. Problem Statement

- Smart-city retail kiosks are deployed in different environments such as hospitals, metro stations, university campuses, and disaster relief zones.
- Although the hardware platform may be similar, each environment has different product requirements, pricing rules, purchase policies, and operational constraints.
- A monolithic kiosk system becomes difficult to maintain, extend, and adapt when new environments, failure handling, or pricing behavior are introduced.
- Therefore, the problem is to design a modular and adaptive kiosk platform using object-oriented principles and design patterns.

2. Proposed Solution

- To solve this problem, we developed **Zephyrus Retail OS**, a modular kiosk platform implemented in C++.
- The system supports four kiosk environments:
  - Hospital
  - Metro
  - University
  - Disaster Relief.
- Each environment is created with its own products, pricing behavior, and policy rules.
- The system supports both user-side and admin-side operations such as purchasing products, purchasing bundles, restocking inventory, running diagnostics, switching system modes, switching kiosk environments, and saving system data.
- The project is built using object-oriented design principles and multiple design patterns including Facade, Strategy, Observer, Command, State, Abstract Factory, Chain of Responsibility, Memento, Singleton, and Mediator.
- This makes the system modular, extensible, and easier to maintain.

3. Programming Language Used

The project was implemented in **C++** because it strongly supports object-oriented programming concepts such as classes, inheritance, polymorphism, abstraction, and encapsulation, which were central to this project.

4. OOP Concepts Used

CONCEPT	HOW USED?
Encapsulation	product stock and pricing data hidden inside classes
Abstraction	subsystems exposed through simple interfaces
Inheritance	pricing strategies, kiosk states, factories, events
Polymorphism	different pricing/state/factory objects used through base types
Low Coupling	subsystems interact through interfaces, mediator, and event bus
Modularity	each feature placed in a separate subsystem/folder

5. Design Patterns Implemented

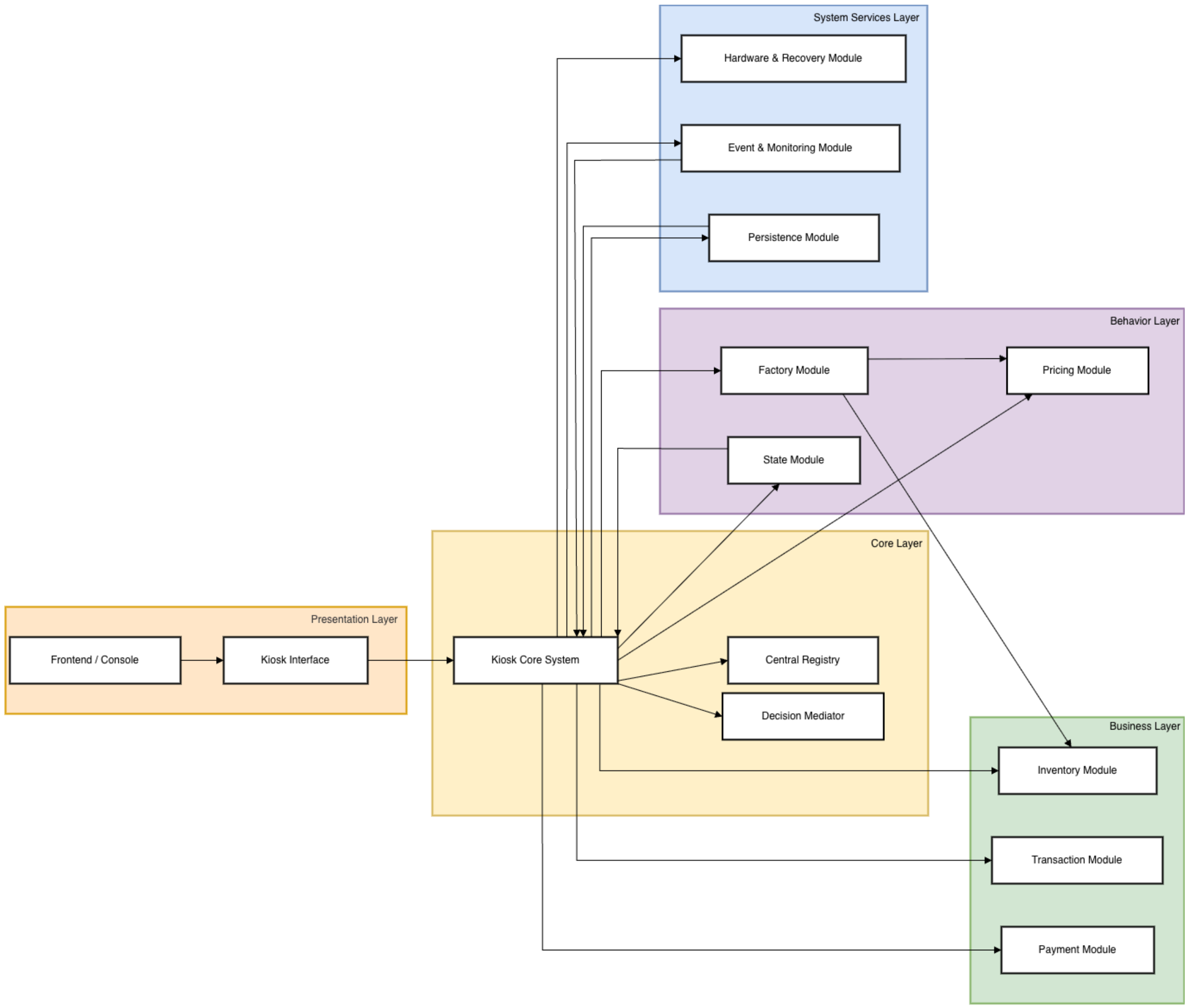
The project uses multiple design patterns to improve extensibility and maintainability.

PATTERN	WHERE IMPLEMENTED?	HOW IT WORKS?
Facade Pattern	KioskInterface	**KioskInterface** provides a simplified entry point for user and admin operations such as purchase, bundle purchase, restock, diagnostics, and transaction history display. Instead of directly interacting with many subsystems, the client interacts with one unified interface.
Strategy Pattern	PricingStrategy StandardPricing DiscountPricing EmergencyPricing PricingSystem	Different pricing behaviors are implemented as interchangeable strategies. The pricing system selects the correct pricing logic depending on the kiosk environment. For example, university kiosks use discount pricing, while disaster relief kiosks use emergency pricing.
Observer Pattern	EventBus Event EventListener CityMonitoringSystem event classes	When important system events occur, such as low stock, hardware failure, or policy violation, the **EventBus** publishes the event and subscribed listeners such as **CityMonitoringSystem** receive notifications automatically.
Command Pattern	Command PurchaseCommand RefundCommand RestockCommand	Operations are encapsulated as command objects so they can be executed in a clean and modular way. This improves separation between interface requests and business operation logic.
State Pattern	KioskState ActiveState PowerSavingState MaintenanceState EmergencyLockdownState	The kiosk behaves differently depending on its current operational mode. For example, purchases may be blocked in maintenance or emergency lockdown state.
Abstract Factory Pattern	KioskFactory HospitalKioskFactory MetroKioskFactory UniversityKioskFactory DisasterReliefKioskFactory	Each factory creates environment-specific inventory and pricing setup without requiring changes in the core logic. This allows the same system to support multiple kiosk environments cleanly.

Chain of Responsibility Pattern	FailureHandler RetryHandler RecalibrationHandler TechnicianAlertHandler	When a hardware failure occurs, the issue is passed through a sequence of handlers. Each handler attempts recovery or escalation before passing it to the next.
Memento Pattern	TransactionSnapshot TransactionCaretaker	Before risky operations, a snapshot of an important state is stored. This supports safe recovery and future rollback extension.
Singleton Pattern	CentralRegistry	**CentralRegistry** ensures that there is one shared source of truth for active kiosk type and current system mode.
Mediator Pattern	DecisionMediator	The mediator helps coordinate state-based operational decisions without tightly coupling modules directly to one another.

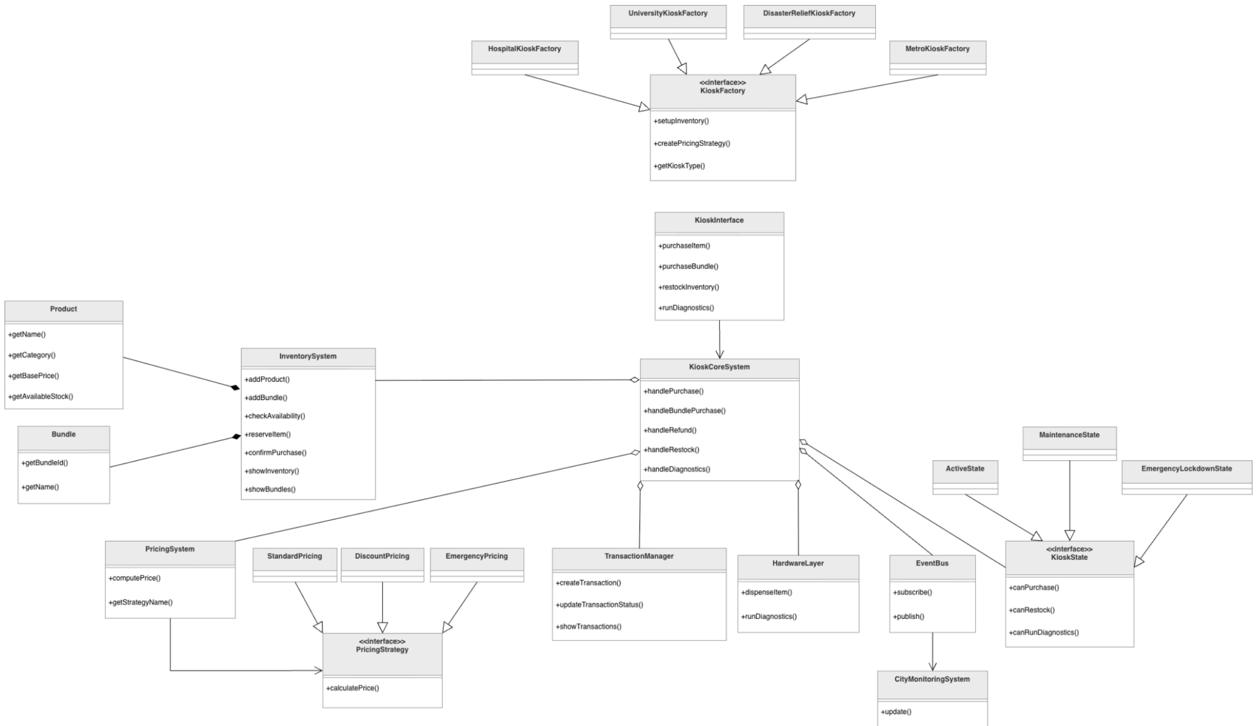
6. Architecture & UML Diagrams

6.1. Architecture



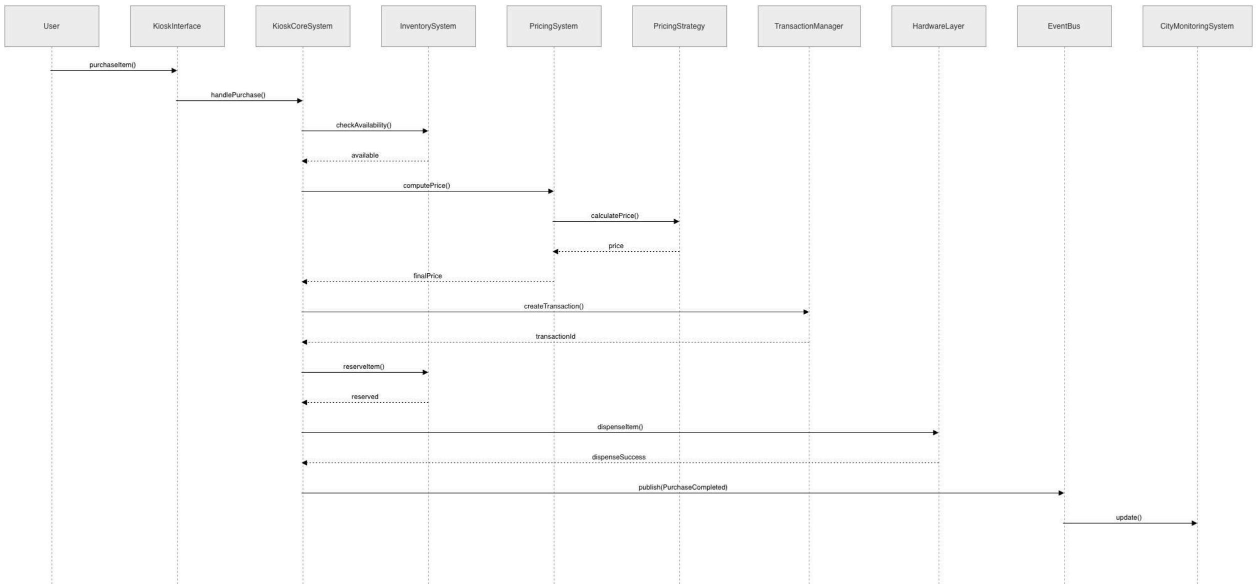
Link: [Architecture]()

6.2. Class Diagram



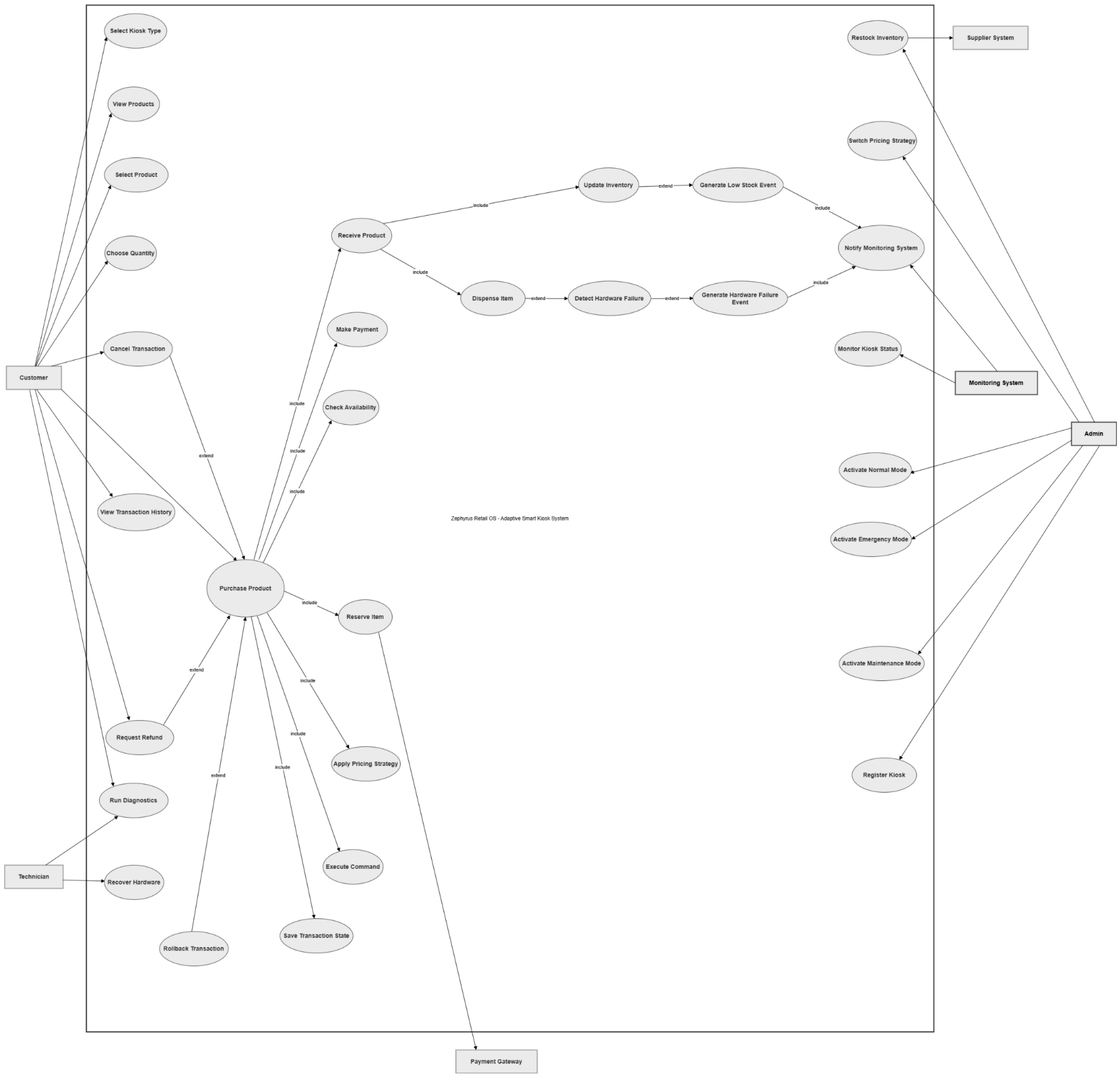
Link: [Class Diagram]

6.3. Sequence Diagram



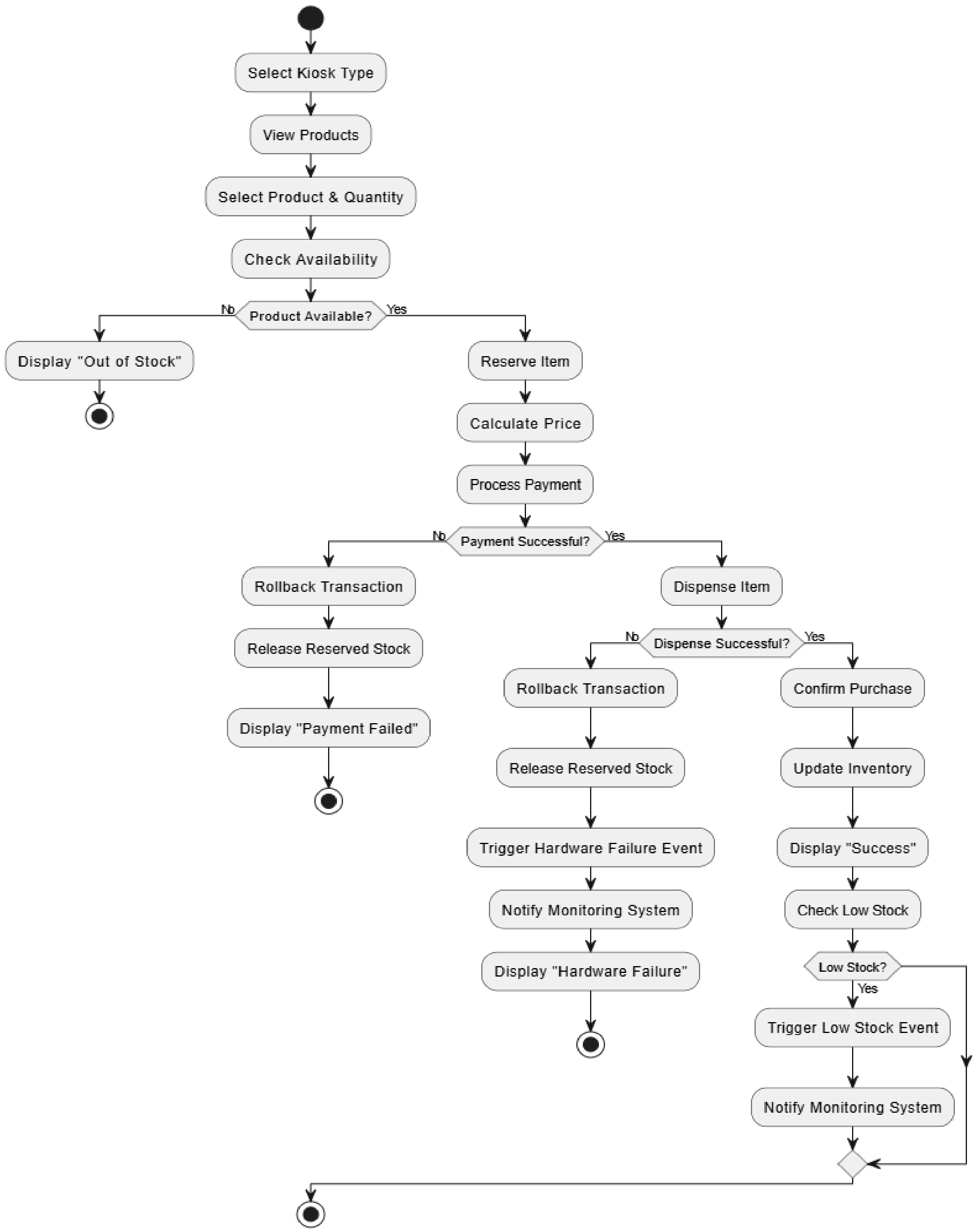
Link: Sequence Diagram

6.4. Use Case Diagram



Link: Use Case

6.5. Activity Diagram



Link: [Activity Diagram]

7. Simulation Demonstration

The project includes a fully working **console-based simulation**. At runtime, the operator selects one of the four kiosk environments and interacts through either User Mode or Admin Mode.

User Mode Demonstrates	Admin Mode Demonstrates
• Browsing available products and bundles • Completing individual and bundle purchases • Triggering simulated hardware failure • Observing automatic recovery flow and State transitions • Dynamic pricing adjustments per environment • Low-stock alerts via Observer notifications	• Secure password-protected login • Inventory restocking and diagnostics execution • Viewing complete transaction history • Switching system mode (Normal / Maintenance / Emergency) • Switching active kiosk environment at runtime • Saving and loading persistence files via Memento

- **Compile & Run**

```
→  LocalOOP g++ -std=c++17 main.cpp \
src/models/Product.cpp \
src/models/Transaction.cpp \
src/inventory/InventorySystem.cpp \
src/inventory/InventoryPolicy.cpp \
src/inventory/Bundle.cpp \
src/payment/PaymentSystem.cpp \
src/pricing/PricingSystem.cpp \
src/pricing/StandardPricing.cpp \
src/pricing/DiscountPricing.cpp \
src/pricing/EmergencyPricing.cpp \
src/hardware/HardwareLayer.cpp \
src/hardware/FailureHandler.cpp \
src/events/EventBus.cpp \
src/monitoring/CityMonitoringSystem.cpp \
src/core/KioskCoreSystem.cpp \
src/core/DecisionMediator.cpp \
src/core/CentralRegistry.cpp \
src/interface/KioskInterface.cpp \
src/commands/PurchaseCommand.cpp \
src/commands/RestockCommand.cpp \
src/commands/RefundCommand.cpp \
src/transaction/TransactionManager.cpp \
src/transaction/TransactionSnapshot.cpp \
src/transaction/TransactionCaretaker.cpp \
src/state/ActiveState.cpp \
src/state/PowerSavingState.cpp \
src/state/MaintenanceState.cpp \
src/state/EmergencyLockdownState.cpp \
src/factory/HospitalKioskFactory.cpp \
src/factory/MetroKioskFactory.cpp \
src/factory/UniversityKioskFactory.cpp \
src/factory/DisasterReliefKioskFactory.cpp \
src/persistence/InventoryStorage.cpp \
src/persistence/TransactionStorage.cpp \
src/persistence/ConfigStorage.cpp \
-Iinclude -o kiosk_app
→  LocalOOP ./kiosk_app
===== Zephyrus Retail OS =====
Select kiosk type:
1. Hospital
2. Metro
3. University
4. Disaster Relief
Enter choice:
```

- **Scenario 1 — System boot + environment setup**

```
→  LocalOOP ./kiosk_app
====== Zephyrus Retail OS ======
Select kiosk type:
1. Hospital
2. Metro
3. University
4. Disaster Relief
Enter choice: 3

[BOOT] Active kiosk environment: University
[BOOT] Initial system mode: ACTIVE

===== MAIN MENU =====
1. User Mode
2. Admin Mode
0. Exit
Enter choice:
```

- **Scenario 2 — User purchase with pricing logic**

```
===== MAIN MENU =====
1. User Mode
2. Admin Mode
0. Exit
Enter choice: 1

===== USER MODE =====
1. Show Products
2. Simulate Hardware Failure on Next Dispense
3. Purchase Product
4. Show Bundles
5. Purchase Bundle
0. Back
Enter option: 1

Available Products:
Product ID: 301 | Name: Earphones | Category: Electronics | Price: 500 | Available Stock: 6
Product ID: 302 | Name: Notebook | Category: Stationery | Price: 40 | Available Stock: 12
Product ID: 303 | Name: Charger | Category: Electronics | Price: 700 | Available Stock: 5
```

```
===== USER MODE =====
1. Show Products
2. Simulate Hardware Failure on Next Dispense
3. Purchase Product
4. Show Bundles
5. Purchase Bundle
0. Back
Enter option: 3

Available Products:
Product ID: 301 | Name: Earphones | Category: Electronics | Price: 500 | Available Stock: 6
Product ID: 302 | Name: Notebook | Category: Stationery | Price: 40 | Available Stock: 12
Product ID: 303 | Name: Charger | Category: Electronics | Price: 700 | Available Stock: 5

Enter product ID: 301
Enter quantity: 2
Enter payment method (UPI/Card/Wallet): UPI
[CORE] Purchase request received.
[STATUS] Kiosk Type: University
[STATUS] Current Mode: ACTIVE
[ENVIRONMENT] University kiosk supports electronics and stationery with student-friendly behavior.
[PRICING] Strategy Applied: DiscountPricing
[PRICING] Base price: 500 x 2 = 1000
[PRICING] Student discount applied: 10%
[PRICING] Discount amount: 100
[PRICING] Final price: 900
[PAYMENT] Processing UPI payment of amount: 900
[PAYMENT] Payment successful.
[HARDWARE] Dispensing product 301 x2
[HARDWARE] Dispensing successful.
[CORE] Purchase completed successfully.

===== TRANSACTION SUMMARY =====
Transaction ID: 1
Kiosk Type: University
Mode: ACTIVE
Product: Earphones
Category: Electronics
Quantity: 2
Payment Method: UPI
Total Price: 900
Status: SUCCESS
===============================
```

- **Scenario 3 — Bundle purchase**

```
○ → LocalOOP ./kiosk_app
===== Zephyrus Retail OS =====
Select kiosk type:
1. Hospital
2. Metro
3. University
4. Disaster Relief
Enter choice: 4

[BOOT] Active kiosk environment: Disaster Relief
[BOOT] Initial system mode: ACTIVE

===== MAIN MENU =====
1. User Mode
2. Admin Mode
0. Exit
Enter choice: 1

===== USER MODE =====
1. Show Products
2. Simulate Hardware Failure on Next Dispense
3. Purchase Product
4. Show Bundles
5. Purchase Bundle
0. Back
Enter option: 4

Available Bundles:
Bundle ID: 903 | Name: Emergency Support Pack | Base Price Total: 510
```

```
===== USER MODE =====
1. Show Products
2. Simulate Hardware Failure on Next Dispense
3. Purchase Product
4. Show Bundles
5. Purchase Bundle
0. Back
Enter option: 5

Available Bundles:
Bundle ID: 903 | Name: Emergency Support Pack | Base Price Total: 510

Enter bundle ID: 903
Enter payment method (UPI/Card/Wallet): Card
[CORE] Bundle purchase request received.
[STATUS] Kiosk Type: Disaster Relief
[STATUS] Current Mode: ACTIVE
[PRICING] Strategy Applied: EmergencyPricing
[PRICING] Base price: 510 x 1 = 510
[PRICING] Emergency relief pricing applied: 20% reduction
[PRICING] Discount amount: 102
[PRICING] Final bundle price: 408
[PAYMENT] Processing Card payment of amount: 408
[PAYMENT] Payment successful.

===== BUNDLE TRANSACTION SUMMARY =====
Transaction ID: 1
Kiosk Type: Disaster Relief
Bundle: Emergency Support Pack
Payment Method: Card
Final Price: 408
Status: SUCCESS
======================================
```

- **Scenario 4 — Hardware failure + recovery**

```
○ ➜  LocalOOP ./kiosk_app
===== Zephyrus Retail OS =====
Select kiosk type:
1. Hospital
2. Metro
3. University
4. Disaster Relief
Enter choice: 2

[BOOT] Active kiosk environment: Metro
[BOOT] Initial system mode: ACTIVE

===== MAIN MENU =====
1. User Mode
2. Admin Mode
0. Exit
Enter choice: 1

===== USER MODE =====
1. Show Products
2. Simulate Hardware Failure on Next Dispense
3. Purchase Product
4. Show Bundles
5. Purchase Bundle
0. Back
Enter option: 2
[TEST] Next dispense will fail.
```

```
===== USER MODE =====
1. Show Products
2. Simulate Hardware Failure on Next Dispense
3. Purchase Product
4. Show Bundles
5. Purchase Bundle
0. Back
Enter option: 3

Available Products:
Product ID: 201 | Name: Water Bottle | Category: Beverage | Price: 20 | Available Stock: 15
Product ID: 202 | Name: Snack Pack | Category: Food | Price: 35 | Available Stock: 10
Product ID: 203 | Name: Umbrella | Category: Travel | Price: 150 | Available Stock: 4

Enter product ID: 202
Enter quantity: 3
Enter payment method (UPI/Card/Wallet): UPI
[CORE] Purchase request received.
[STATUS] Kiosk Type: Metro
[STATUS] Current Mode: ACTIVE
[ENVIRONMENT] Metro kiosk supports quick daily-use purchases with lightweight policy limits.
[PRICING] Strategy Applied: StandardPricing
[PRICING] Base price: 35 x 3 = 105
[PRICING] No additional adjustment applied
[PRICING] Final price: 105
[PAYMENT] Processing UPI payment of amount: 105
[PAYMENT] Payment successful.
[HARDWARE] Dispensing product 202 x3
[HARDWARE] Dispensing failed.
[RECOVERY] RetryHandler processing: Dispense failure for product ID 202
[RECOVERY] RecalibrationHandler processing: Dispense failure for product ID 202
[RECOVERY] TechnicianAlertHandler processing: Dispense failure for product ID 202
[RECOVERY] Technician has been notified.
[CITY MONITORING] HardwareFailureEvent -> Dispensing failed for product ID 202. Reserved stock released safely. [Action: maintenance unit should inspect
kiosk]
[RECOVERY] Hardware failure handled safely.
```

- # Scenario 5 — Policy violation

```
→ LocalOOP ./kiosk_app
===== Zephyrus Retail OS =====
Select kiosk type:
1. Hospital
2. Metro
3. University
4. Disaster Relief
Enter choice: 1

[BOOT] Active kiosk environment: Hospital
[BOOT] Initial system mode: ACTIVE

===== MAIN MENU =====
1. User Mode
2. Admin Mode
0. Exit
Enter choice: 1

===== USER MODE =====
1. Show Products
2. Simulate Hardware Failure on Next Dispense
3. Purchase Product
4. Show Bundles
5. Purchase Bundle
0. Back
Enter option: 3

Available Products:
Product ID: 101 | Name: Paracetamol | Category: Medicine | Price: 20 | Available Stock: 10
Product ID: 102 | Name: Prescription Medicine | Category: Medicine | Price: 120 | Available Stock: 5
Product ID: 103 | Name: Sanitizer | Category: Hygiene | Price: 50 | Available Stock: 8

Enter product ID: 102
Enter quantity: 3
Enter payment method (UPI/Card/Wallet): Wallet
[CORE] Purchase request received.
[STATUS] Kiosk Type: Hospital
[STATUS] Current Mode: ACTIVE
[ENVIRONMENT] Hospital kiosk handles medical essentials and restricted prescription items.
[POLICY] Hospital kiosk allows maximum 2 units for restricted prescription items.
[CITY MONITORING] PolicyViolationEvent -> Hospital kiosk allows maximum 2 units for restricted prescription items. [Action: rule enforcement triggered]
```

```
→ LocalOOP ./kiosk_app
===== Zephyrus Retail OS =====
Select kiosk type:
1. Hospital
2. Metro
3. University
4. Disaster Relief
Enter choice: 4

[BOOT] Active kiosk environment: Disaster Relief
[BOOT] Initial system mode: ACTIVE

===== MAIN MENU =====
1. User Mode
2. Admin Mode
0. Exit
Enter choice: 1

===== USER MODE =====
1. Show Products
2. Simulate Hardware Failure on Next Dispense
3. Purchase Product
4. Show Bundles
5. Purchase Bundle
0. Back
Enter option: 3

Available Products:
Product ID: 401 | Name: Emergency Kit | Category: Relief | Price: 250 | Available Stock: 6
Product ID: 402 | Name: Water Pack | Category: Relief | Price: 80 | Available Stock: 10
Product ID: 403 | Name: First Aid Box | Category: Relief | Price: 180 | Available Stock: 5

Enter product ID: 402
Enter quantity: 5
Enter payment method (UPI/Card/Wallet): Card
[CORE] Purchase request received.
[STATUS] Kiosk Type: Disaster Relief
[STATUS] Current Mode: ACTIVE
[ENVIRONMENT] Disaster Relief kiosk prioritizes essential items with strict emergency purchase limits.
[POLICY] Disaster Relief kiosk allows maximum 2 essential items per transaction.
[CITY MONITORING] PolicyViolationEvent -> Disaster Relief kiosk allows maximum 2 essential items per transaction. [Action: rule enforcement triggered]
```

- **Scenario 6 — Admin mode operations**

```
→  LocalOOP ./kiosk_app
===== Zephyrus Retail OS =====
Select kiosk type:
1. Hospital
2. Metro
3. University
4. Disaster Relief
Enter choice: 1

[BOOT] Active kiosk environment: Hospital
[BOOT] Initial system mode: ACTIVE

===== MAIN MENU =====
1. User Mode
2. Admin Mode
0. Exit
Enter choice: 2

Enter admin password: 1234
[ADMIN] Login successful.
```

```
===== ADMIN MODE =====
1. Show Inventory
2. Restock Item
3. Run Diagnostics
4. Show Transaction History
5. Change System Mode
6. Switch Kiosk Environment
7. Save Inventory Data
8. Save Transaction History
9. Save Current Config
0. Back
Enter option: 7
[PERSISTENCE] Inventory saved to data/inventory.csv

===== ADMIN MODE =====
1. Show Inventory
2. Restock Item
3. Run Diagnostics
4. Show Transaction History
5. Change System Mode
6. Switch Kiosk Environment
7. Save Inventory Data
8. Save Transaction History
9. Save Current Config
0. Back
Enter option: 2

Available Products:
Product ID: 101 | Name: Paracetamol | Category: Medicine | Price: 20 | Available Stock: 10
Product ID: 102 | Name: Prescription Medicine | Category: Medicine | Price: 120 | Available Stock: 5
Product ID: 103 | Name: Sanitizer | Category: Hygiene | Price: 50 | Available Stock: 8

Enter product ID to restock: 103
Enter quantity to add: 12
[ADMIN] Product 103 restocked by 12 units. Transaction ID: 1
```

```
===== ADMIN MODE =====
1. Show Inventory
2. Restock Item
3. Run Diagnostics
4. Show Transaction History
5. Change System Mode
6. Switch Kiosk Environment
7. Save Inventory Data
8. Save Transaction History
9. Save Current Config
0. Back
Enter option: 3
[STATUS] Running diagnostics for Hospital kiosk in mode ACTIVE...
[HARDWARE] Diagnostics completed. All systems normal.

===== ADMIN MODE =====
1. Show Inventory
2. Restock Item
3. Run Diagnostics
4. Show Transaction History
5. Change System Mode
6. Switch Kiosk Environment
7. Save Inventory Data
8. Save Transaction History
9. Save Current Config
0. Back
Enter option: 4

===== TRANSACTION HISTORY =====
ID: 1 | Type: RESTOCK | Product ID: 103 | Qty: 12 | Amount: 0 | Method: ADMIN | Status: SUCCESS
```

```
===== ADMIN MODE =====
1. Show Inventory
2. Restock Item
3. Run Diagnostics
4. Show Transaction History
5. Change System Mode
6. Switch Kiosk Environment
7. Save Inventory Data
8. Save Transaction History
9. Save Current Config
0. Back
Enter option: 2

Available Products:
Product ID: 101 | Name: Paracetamol | Category: Medicine | Price: 20 | Available Stock: 10
Product ID: 102 | Name: Prescription Medicine | Category: Medicine | Price: 120 | Available Stock: 5
Product ID: 103 | Name: Sanitizer | Category: Hygiene | Price: 50 | Available Stock: 20

Enter product ID to restock: 102
Enter quantity to add: 12
[ADMIN] Product 102 restocked by 12 units. Transaction ID: 2

===== ADMIN MODE =====
1. Show Inventory
2. Restock Item
3. Run Diagnostics
4. Show Transaction History
5. Change System Mode
6. Switch Kiosk Environment
7. Save Inventory Data
8. Save Transaction History
9. Save Current Config
0. Back
Enter option: 8
[PERSISTENCE] Transactions saved to data/transactions.csv

===== ADMIN MODE =====
1. Show Inventory
2. Restock Item
3. Run Diagnostics
4. Show Transaction History
5. Change System Mode
6. Switch Kiosk Environment
7. Save Inventory Data
8. Save Transaction History
9. Save Current Config
0. Back
Enter option: 9
[PERSISTENCE] Config saved to data/config.txt
```

- ## Scenario 7 — Mode switching

```
➜  LocalOOP ./kiosk_app
===== Zephyrus Retail OS =====
Select kiosk type:
1. Hospital
2. Metro
3. University
4. Disaster Relief
Enter choice: 4

[BOOT] Active kiosk environment: Disaster Relief
[BOOT] Initial system mode: ACTIVE

===== MAIN MENU =====
1. User Mode
2. Admin Mode
0. Exit
Enter choice: 2

Enter admin password: 1234
[ADMIN] Login successful.

===== ADMIN MODE =====
1. Show Inventory
2. Restock Item
3. Run Diagnostics
4. Show Transaction History
5. Change System Mode
6. Switch Kiosk Environment
7. Save Inventory Data
8. Save Transaction History
9. Save Current Config
0. Back
Enter option: 5

Select system mode:
1. Active
2. Power Saving
3. Maintenance
4. Emergency Lockdown
Enter choice: 4
[ADMIN] System mode changed to: EMERGENCY_LOCKDOWN
```

```
===== USER MODE =====
1. Show Products
2. Simulate Hardware Failure on Next Dispense
3. Purchase Product
4. Show Bundles
5. Purchase Bundle
0. Back
Enter option: 3

Available Products:
Product ID: 401 | Name: Emergency Kit | Category: Relief | Price: 250 | Available Stock: 6
Product ID: 402 | Name: Water Pack | Category: Relief | Price: 80 | Available Stock: 10
Product ID: 403 | Name: First Aid Box | Category: Relief | Price: 180 | Available Stock: 5

Enter product ID: 402
Enter quantity: 5
Enter payment method (UPI/Card/Wallet): Card
[CORE] Purchase request received.
[STATUS] Kiosk Type: Disaster Relief
[STATUS] Current Mode: EMERGENCY_LOCKDOWN
[ENVIRONMENT] Disaster Relief kiosk prioritizes essential items with strict emergency purchase limits.
[STATE] Purchase blocked because current mode is EMERGENCY_LOCKDOWN
[CITY MONITORING] PolicyViolationEvent -> Purchase blocked because current mode is EMERGENCY_LOCKDOWN [Action: rule enforcement triggered]
```

- **Scenario 8 — Environment switching without rerun**

```
➜  LocalOOP ./kiosk_app
2. Metro
3. University
4. Disaster Relief
Enter choice: 2

[BOOT] Active kiosk environment: Metro
[BOOT] Initial system mode: ACTIVE

===== MAIN MENU =====
1. User Mode
2. Admin Mode
0. Exit
Enter choice: 2

Enter admin password: 1234
[ADMIN] Login successful.

===== ADMIN MODE =====
1. Show Inventory
2. Restock Item
3. Run Diagnostics
4. Show Transaction History
5. Change System Mode
6. Switch Kiosk Environment
7. Save Inventory Data
8. Save Transaction History
9. Save Current Config
0. Back
Enter option: 6

Select new kiosk type:
1. Hospital
2. Metro
3. University
4. Disaster Relief
Enter choice: 3
[ADMIN] Kiosk environment switched to: University
[ADMIN] Active pricing strategy will now follow the selected environment.
```

```
===== USER MODE =====
1. Show Products
2. Simulate Hardware Failure on Next Dispense
3. Purchase Product
4. Show Bundles
5. Purchase Bundle
0. Back
Enter option: 3

Available Products:
Product ID: 301 | Name: Earphones | Category: Electronics | Price: 500 | Available Stock: 6
Product ID: 302 | Name: Notebook | Category: Stationery | Price: 40 | Available Stock: 12
Product ID: 303 | Name: Charger | Category: Electronics | Price: 700 | Available Stock: 5

Enter product ID: 302
Enter quantity: 3
Enter payment method (UPI/Card/Wallet): Card
[CORE] Purchase request received.
[STATUS] Kiosk Type: University
[STATUS] Current Mode: ACTIVE
[ENVIRONMENT] University kiosk supports electronics and stationery with student-friendly behavior.
[PRICING] Strategy Applied: DiscountPricing
[PRICING] Base price: 40 x 3 = 120
[PRICING] Student discount applied: 10%
[PRICING] Discount amount: 12
[PRICING] Final price: 108
[PAYMENT] Processing Card payment of amount: 108
[PAYMENT] Payment successful.
[HARDWARE] Dispensing product 302 x3
[HARDWARE] Dispensing successful.
[CORE] Purchase completed successfully.

===== TRANSACTION SUMMARY =====
Transaction ID: 1
Kiosk Type: University
Mode: ACTIVE
Product: Notebook
Category: Stationery
Quantity: 3
Payment Method: Card
Total Price: 108
Status: SUCCESS
===============================
```

- **Scenario 9 — Persistence**

```
===== ADMIN MODE =====
1. Show Inventory
2. Restock Item
3. Run Diagnostics
4. Show Transaction History
5. Change System Mode
6. Switch Kiosk Environment
7. Save Inventory Data
8. Save Transaction History
9. Save Current Config
0. Back
Enter option: 1

Available Products:
Product ID: 201 | Name: Water Bottle | Category: Beverage | Price: 20 | Available Stock: 15
Product ID: 202 | Name: Snack Pack | Category: Food | Price: 35 | Available Stock: 10
Product ID: 203 | Name: Umbrella | Category: Travel | Price: 150 | Available Stock: 4

===== ADMIN MODE =====
1. Show Inventory
2. Restock Item
3. Run Diagnostics
4. Show Transaction History
5. Change System Mode
6. Switch Kiosk Environment
7. Save Inventory Data
8. Save Transaction History
9. Save Current Config
0. Back
Enter option: 2

Available Products:
Product ID: 201 | Name: Water Bottle | Category: Beverage | Price: 20 | Available Stock: 15
Product ID: 202 | Name: Snack Pack | Category: Food | Price: 35 | Available Stock: 10
Product ID: 203 | Name: Umbrella | Category: Travel | Price: 150 | Available Stock: 4

Enter product ID to restock: 201
Enter quantity to add: 5
[ADMIN] Product 201 restocked by 5 units. Transaction ID: 1
```

```
===== ADMIN MODE =====
1. Show Inventory
2. Restock Item
3. Run Diagnostics
4. Show Transaction History
5. Change System Mode
6. Switch Kiosk Environment
7. Save Inventory Data
8. Save Transaction History
9. Save Current Config
0. Back
Enter option: 2

Available Products:
Product ID: 201 | Name: Water Bottle | Category: Beverage | Price: 20 | Available Stock: 20
Product ID: 202 | Name: Snack Pack | Category: Food | Price: 35 | Available Stock: 10
Product ID: 203 | Name: Umbrella | Category: Travel | Price: 150 | Available Stock: 4

Enter product ID to restock: 203
Enter quantity to add: 14
[ADMIN] Product 203 restocked by 14 units. Transaction ID: 2

===== ADMIN MODE =====
1. Show Inventory
2. Restock Item
3. Run Diagnostics
4. Show Transaction History
5. Change System Mode
6. Switch Kiosk Environment
7. Save Inventory Data
8. Save Transaction History
9. Save Current Config
0. Back
Enter option: 4

===== TRANSACTION HISTORY =====
ID: 1 | Type: RESTOCK | Product ID: 201 | Qty: 5 | Amount: 0 | Method: ADMIN | Status: SUCCESS
ID: 2 | Type: RESTOCK | Product ID: 203 | Qty: 14 | Amount: 0 | Method: ADMIN | Status: SUCCESS
```

```
===== ADMIN MODE =====
1. Show Inventory
2. Restock Item
3. Run Diagnostics
4. Show Transaction History
5. Change System Mode
6. Switch Kiosk Environment
7. Save Inventory Data
8. Save Transaction History
9. Save Current Config
0. Back
Enter option: 8
[PERSISTENCE] Transactions saved to data/transactions.csv

===== ADMIN MODE =====
1. Show Inventory
2. Restock Item
3. Run Diagnostics
4. Show Transaction History
5. Change System Mode
6. Switch Kiosk Environment
7. Save Inventory Data
8. Save Transaction History
9. Save Current Config
0. Back
Enter option: 7
[PERSISTENCE] Inventory saved to data/inventory.csv

===== ADMIN MODE =====
1. Show Inventory
2. Restock Item
3. Run Diagnostics
4. Show Transaction History
5. Change System Mode
6. Switch Kiosk Environment
7. Save Inventory Data
8. Save Transaction History
9. Save Current Config
0. Back
Enter option: 9
[PERSISTENCE] Config saved to data/config.txt
```

LocalOOP > data > transactions.csv > data

```
transactionId,type,productId,quantity,amount,paymentMethod,status
1,RESTOCK,201,5,0,ADMIN,SUCCESS
2,RESTOCK,203,14,0,ADMIN,SUCCESS
```

LocalOOP > data > inventory.csv > data

```
productId,name,category,basePrice,stock,reservedStock,faultyStock,lowStockThreshold,restricted,essential
201,Water Bottle,Beverage,20,20,0,0,3,0,1
202,Snack Pack,Food,35,10,0,0,2,0,1
203,Umbrella,Travel,150,18,0,0,1,0,0
```

LocalOOP > data > config.txt

```
kioskType=Metro
mode=ACTIVE

```

8. GitHub Repository Link

GitHub Repository Link: https://github.com/priiyu12/Aura-Retail-OS/